UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

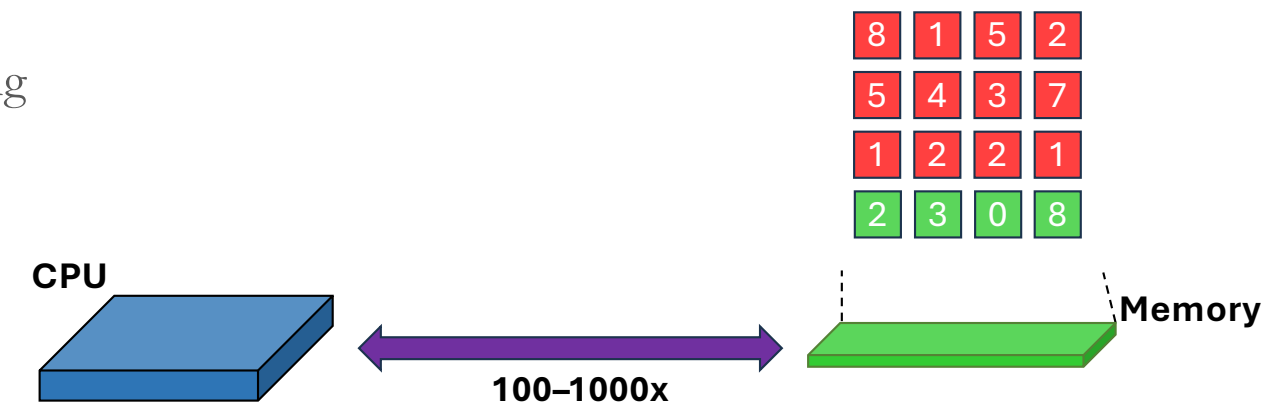
Enabling Near-Accelerator Performance With Generic Processing-Using-Memory Architectures

Ryan Wong

MCCSys@ASPLOS'26

The High Cost of Data Movement

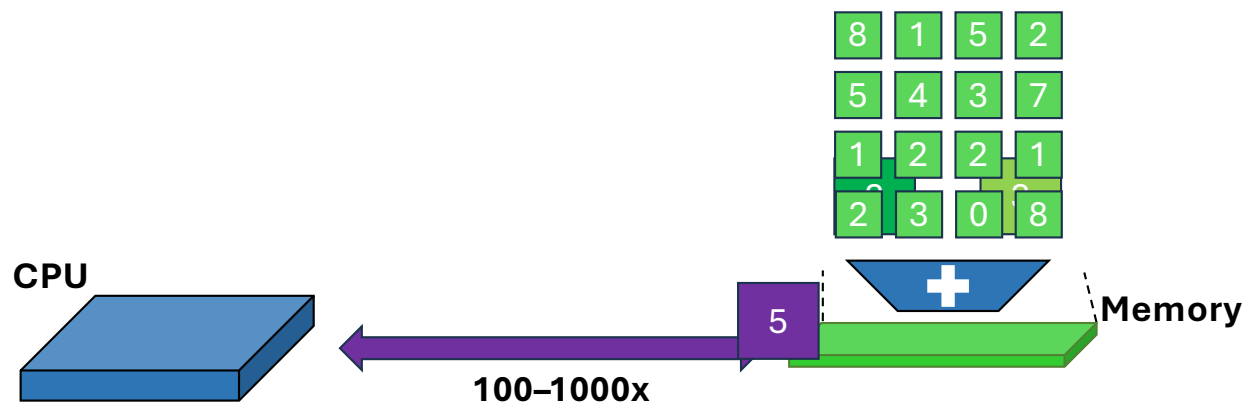
- **Modern computer systems rely on von Neumann architecture**
  - Discrete compute (e.g., CPU) and memory (e.g., RAM)
  - Relies on shuffling data between CPU and memory
- **Data movement overheads eclipse compute w.r.t. time *and* energy**
- **Applications are increasingly data intensive, *exacerbating* these overheads**
  - Machine learning
  - Scientific computing
  - Database systems
  - Graph analytics



Techniques to Reduce Data Movement Overheads

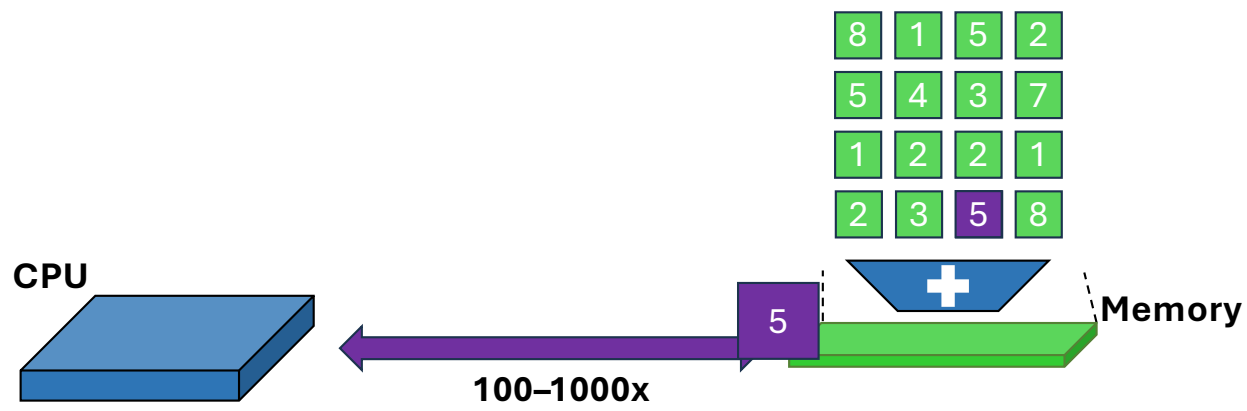
- **Reducing data movement is a major problem today!**
  - Reduce the *cost* (e.g., encoding, circuits)
  - Reduce the *amount* (e.g., offload computation to memory)

- ***Processing-in-memory* (PIM) enables computation in the memory substrate**

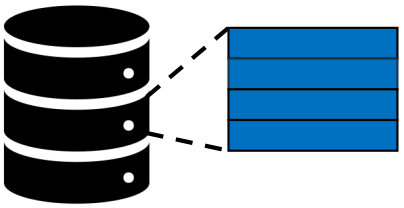


Reducing Data Movement Overheads With PUM

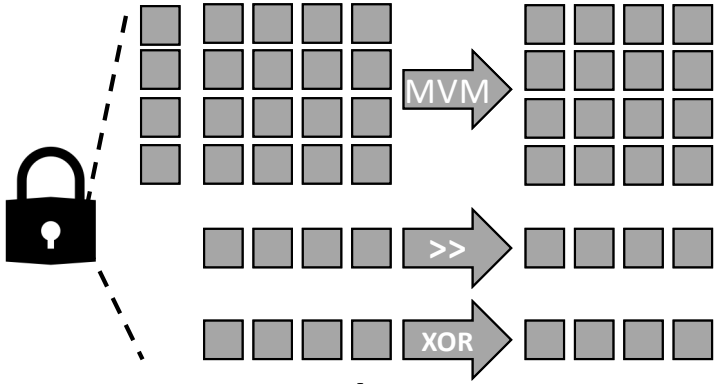
- ***Processing-using-memory*** **(PUM) extends PIM by enabling computation within the memory cells *without* additional logic**
- **PUM can further reduce data movement beyond PIM**



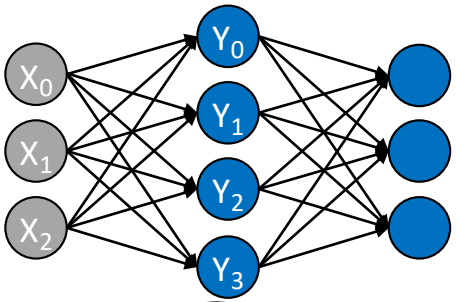
The Trap of Specialization



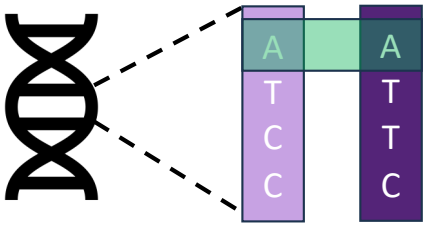
Data Analytics



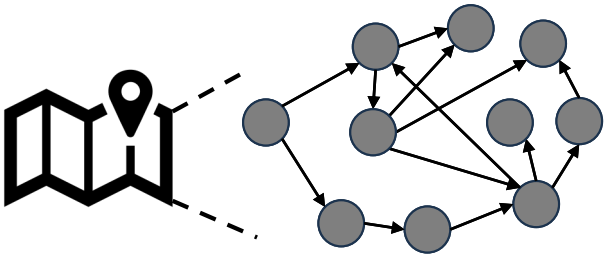
Cryptography



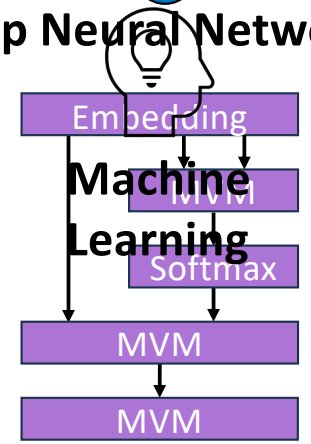
Deep Neural Networks



Genomics



Graph Analytics



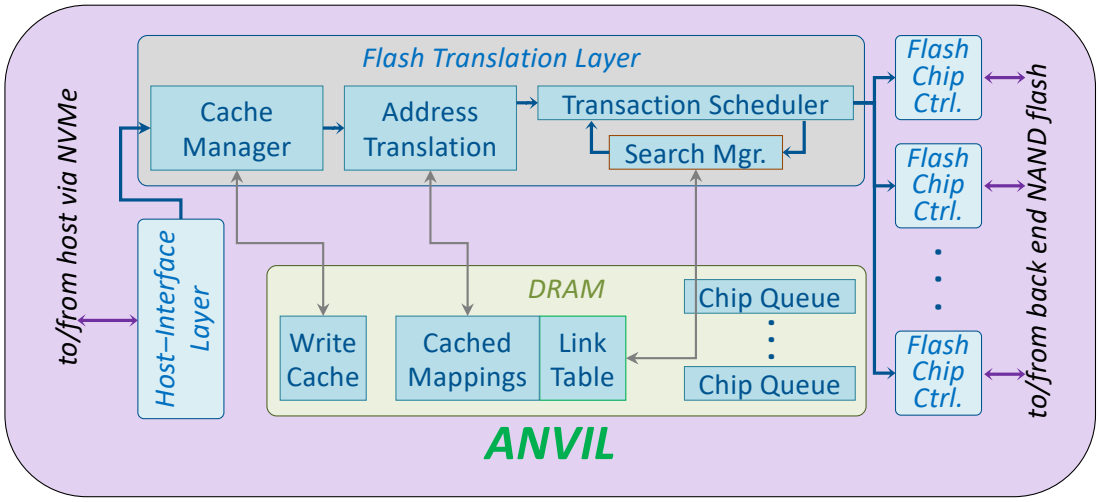
Machine Learning

Large Language Models

New applications require *substantial* re-design of PUM-enabled accelerators!

Approaching Accelerator Benefits Without Specialization

- Target generic abstractions and common kernels across applications
- Re-use existing hardware
- Maintain support for existing operations (*i.e., avoid breaking things*)



Accelerates name–value pair workloads

Implemented on top of a conventional SSD

Reuses NAND flash for PUM and SSD resources

Maintains support for existing I/O functionality

Compatible with state-of-the-art interface protocols (e.g., NVMe)

Speedup: 4x for database transactions, 25x for database analytics, 14.6% speedup for graph workloads

Introduction

ANVIL
(ISCA 2025)

DARTH-PUM: A Preview
(ASPLOS 2026)

Name–Value Pairs as a Generic Abstraction for Data Processing

- **Name–value pairs (NVPs) are widely used in many applications**
  - Highly flexible
  - Broad applicability
    - » Arrays, hashes, dictionaries, content-addressable memory, lookup tables
    - » Key–value pairs, databases, graphs, network routing

ANVIL: An In-Storage Accelerator for Name–Value Data Stores

Ryan Wong, Univ. of Illinois Urbana-Champaign, Urbana, IL, USA; Sandia National Laboratories, Albuquerque, NM, USA; ryanw13@illinois.edu

Nikita Kim, Carnegie Mellon Univ., Pittsburgh, PA, USA; nikitakis@andrew.cmu.edu

Aniket Das, Univ. of Illinois Urbana-Champaign, Urbana, IL, USA; akdas3@illinois.edu

Kevin Higgs, Univ. of Illinois Urbana-Champaign, Urbana, IL, USA; kmhiggs2@illinois.edu

Engin Ipek, Micron, San Diego, CA, USA; eipek@micron.com

Sapan Agarwal, Sandia National Laboratories, Livermore, CA, USA; sagarwa@sandia.gov

Saugata Ghose, Univ. of Illinois Urbana-Champaign, Urbana, IL, USA; ghose@illinois.edu

Ben Feinberg, Sandia National Laboratories, Albuquerque, NM, USA; bfeinbe@sandia.gov

Abstract

Name–value pairs (NVPs) are a widely-used abstraction to organize data in millions of applications. At a high level, an NVP associates a name (e.g., array index, key, hash) with each value in a collection of data. Specific NVP data store formats can vary widely, ranging from simple arrays/dictionaries and lookup tables to key–value stores and data mining workloads. Despite their importance, existing optimizations for NVPs are limited to only a single data store format, as the broad definition of NVPs allows for significant heterogeneity in encoding and implementation.

We propose ANVIL, the first end-to-end system that allows programmers to broadly accelerate most formats of NVPs. With a conventional solid-state drive (SSD), large-scale NVP lookups can saturate both external and internal SSD bandwidth, as every NVP in the data store needs to be sent back to the host CPU to check for a matching name. ANVIL makes use of *in-storage processing* to avoid reading out any data for names that do not match, by performing name match checks directly inside the SSD's NAND flash chips. We demonstrate that ANVIL can substantially reduce disk I/O, reduce metadata overheads, and provide speedups of 4.0×, 25×, and 14.6× over a conventional SSD, for three different NVP workloads (database transactions, analytics, and graph processing).

CCS Concepts

• **Computer systems organization** → **Architectures.** • **Hardware** → **Memory and dense storage.** • **Information systems** → *Record storage alternatives.*

This work is licensed under a Creative Commons Attribution 4.0 International License.

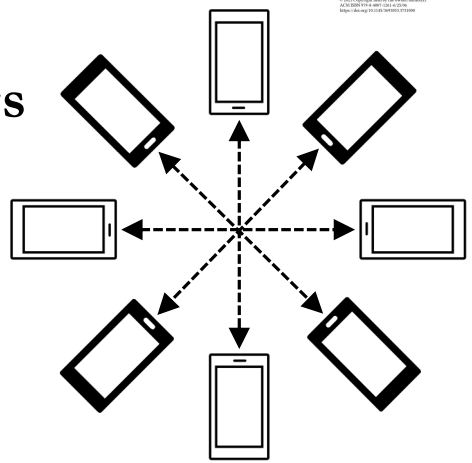
ISCA '20, Tokyo, Japan
© 2020 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-7526-8/20/05
https://doi.org/10.1145/3579853.3731008

1 Introduction

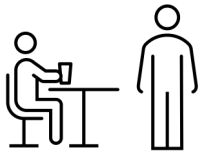
The *name–value pair* (NVP) abstraction is one of the most widely used approaches to organize data in modern programs. Broadly defined, an NVP data store is any collection of data that associates a name (e.g., an array index, a key, a hash index) with each value in the collection, allowing programs to use the associated name to find a value. The concept of NVP can be found in millions of applications in both hardware (e.g., content-addressable memories [52, 53], lookup tables [45]) and software (e.g., array/dictionary data structures [115], data mining [3], key–value store databases [33, 44], web addresses [131], and data formats [16, 46, 4]). As the amount of data generated and consumed by modern applications continues to grow exponentially [9, 28, 35, 38, 59, 62, 117, 119], there has been an increased reliance on sophisticated NVP-based data management systems as the backbone of these applications.

While NVPs have many uses, a common thread across the vast majority of NVP-based applications is the need to retrieve NVPs from the data store whose names match a user-requested query. Although this association-based approach is powerful, the ambiguous definition of names introduces a non-trivial overhead for modern hardware systems. Memory subsystems do not inherently map names to data addresses, and the typical method of accessing an NVP requires some form of indirection. While the most basic form of NVP data stores (e.g., contiguous data arrays) are able to use

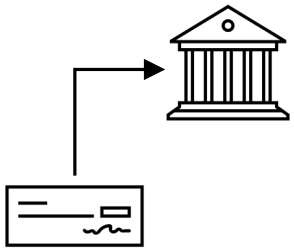
- **Capture interactions or relationships between pairs**



{Name, Time}



{Patron, Order}

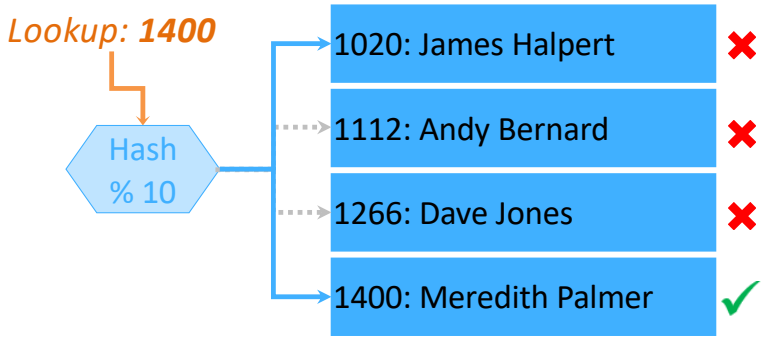


{Recipient, Value}

{Followers, Post}

Finding the "Name" in the Haystack

- **Flexible usage of NVPs adds complexity for memory and storage systems**
  - Modern memory & storage systems use addresses
  - Names add another layer of indirection, or more!
- ***Look up* a student ID**
- ***Scans* are costly, requires data movement from host–memory/host–SSD for *every* NVP**
- **Additional metadata (e.g., hash) can be used to reduce the number of NVPs transferred**

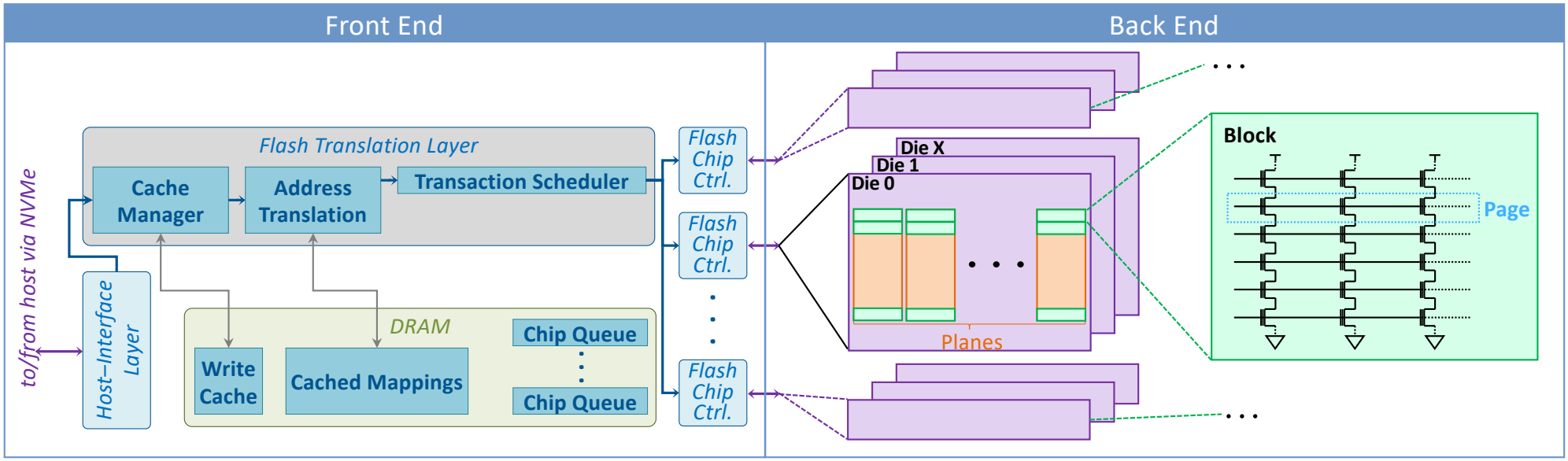


In-Storage Computation for Name–Value Pairs

- **NVP-based workloads can have footprints of GBs to TBs**
  - Solid-state drives (SSDs) can provide capacity
  - SSD access takes µs, requires large data transfers
- **ANVIL: in-SSD name–value pair acceleration**
  - **Processing-using-memory (PUM)** for bulk parallel name lookup in SSD
  - Returns *only* matching NVPs, **reducing both external/internal SSD data movement**
  - General support for NVPs, enabling more sophisticated application (e.g., databases)
  - NVMe-compatible name–value API for programmer support
  - Reliability techniques to guarantee all matches are found/returned
- **Speedup: 4x for database transactions, 25x for database analytics, 14.6% speedup for graph workloads**
- **Published at ISCA 2025**

Solid-State Drives

- **Conventional SSDs**
  - Front end (FE): **flash translation layer** (FTL), DRAM
  - Back end (BE): NAND flash
- **High throughput by employing many levels of parallelism**

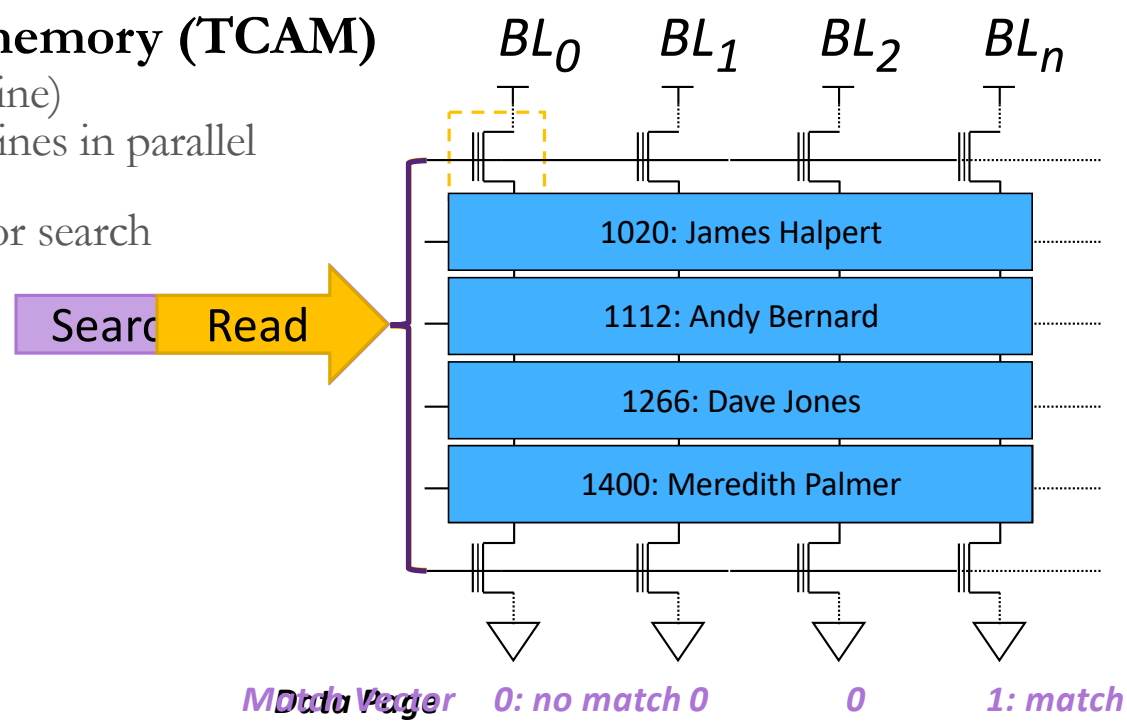


NAND Flash Operations

- **Program state by setting threshold voltage (V_{th}) of transistor (cell)**
  - Data are stored as pages along *wordlines*
  - Read/write are *page* granularity

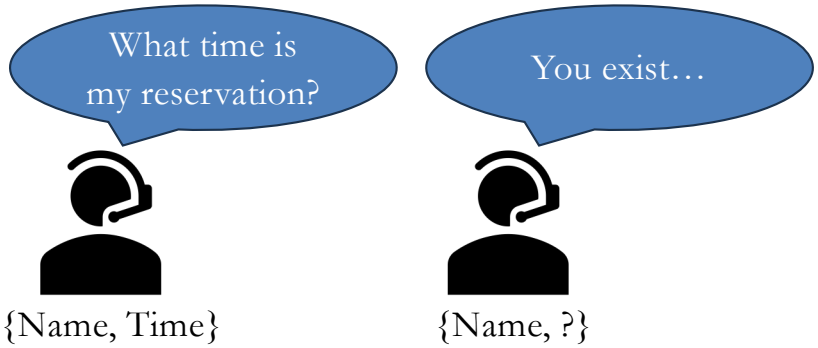
Tseng et al., IEDM 2020

- **IMS: ternary content-addressable memory (TCAM)**
  - Stores data in *vertical* format (i.e., along bitline)
  - Block-level search compares against *all* bitlines in parallel via *processing-using-memory*
  - Returns *match vector*: each bit set is match for search
  - Does ***not*** return stored data



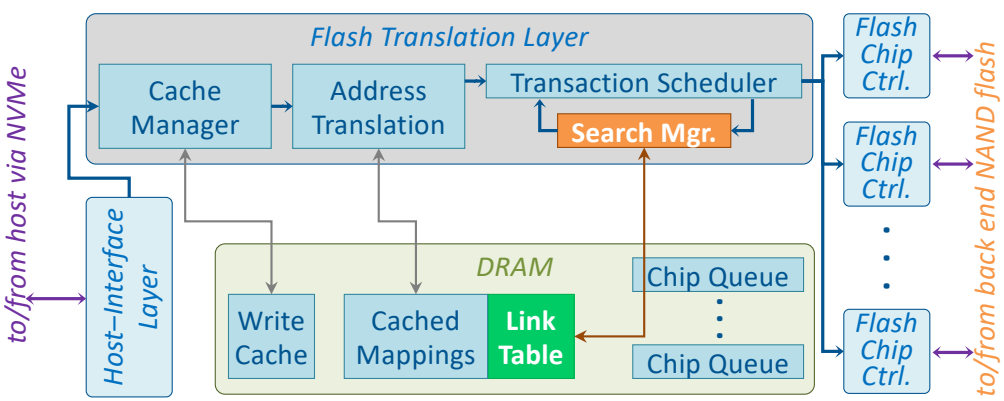
Hope You Find What You're Looking For

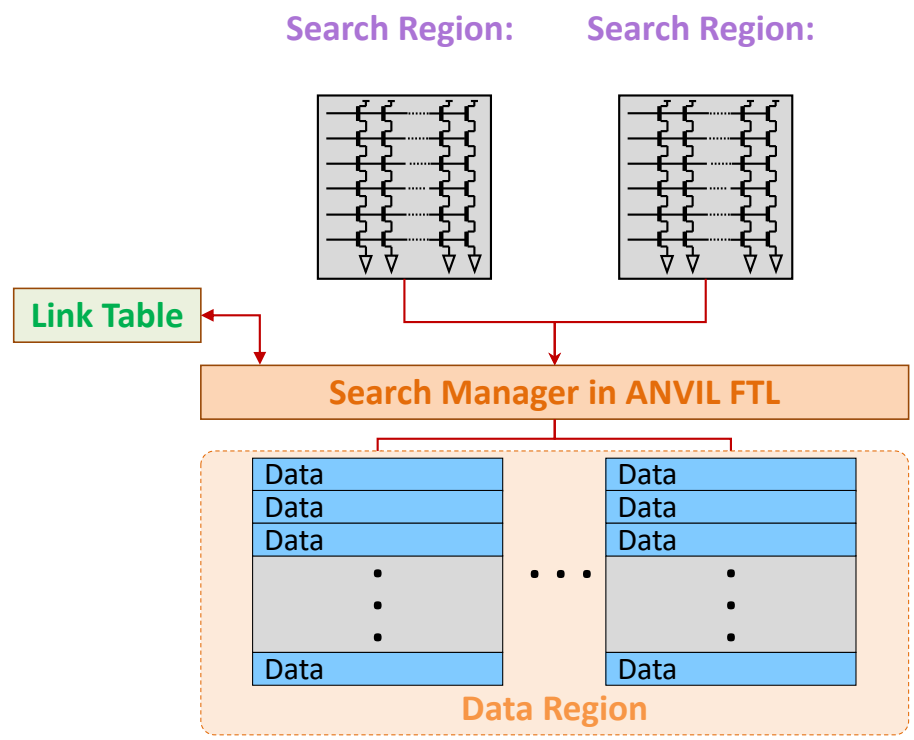
- **IMS can *only* detect a match (i.e., if NVP is in NAND flash)**
- **Storage solution capable of accelerating NVP lookup**
- **Manage conventional I/O requests alongside lookup**
- **Programmer abstractions**
  - Clean interface allowing continued use of NVP abstraction
  - Support more complex applications (e.g., many names to value)



Accelerating Name–Values With In-Storage Lookups

- **Dual data representation for identifying *and* retrieving full NVP**
  - *Search regions (SR):* vertical data
  - *Data regions (DR):* conventional horizontal data
- ***Search manager* inside FTL**
- **Introduce *link table* (in DRAM)**
- **NVMe-compatible interface**





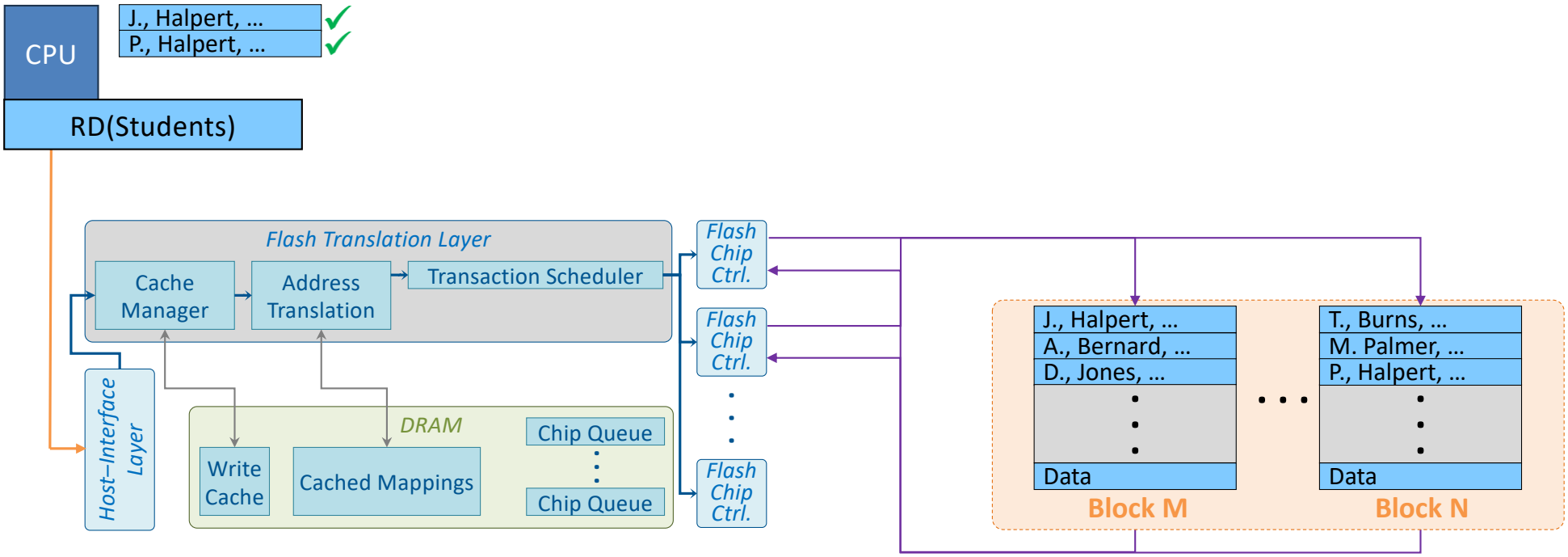
From NVP to Databases

- **NVPs can be used to build more sophisticated databases: {attribute, tuple}**
- **Query: `SELECT {*} FROM {Students} WHERE LastName='Halpert';`**
  - `Name={LastName}, Value = {StudentID, FirstName, LastName, Group}`

StudentID	FirstName	LastName	Group
1020	James	Halpert	3
1112	Andy	Bernard	1
1266	Dave	Jones	2
1322	Tom	Burns	0
1400	Meredith	Palmer	3
1451	Pam	Halpert	1

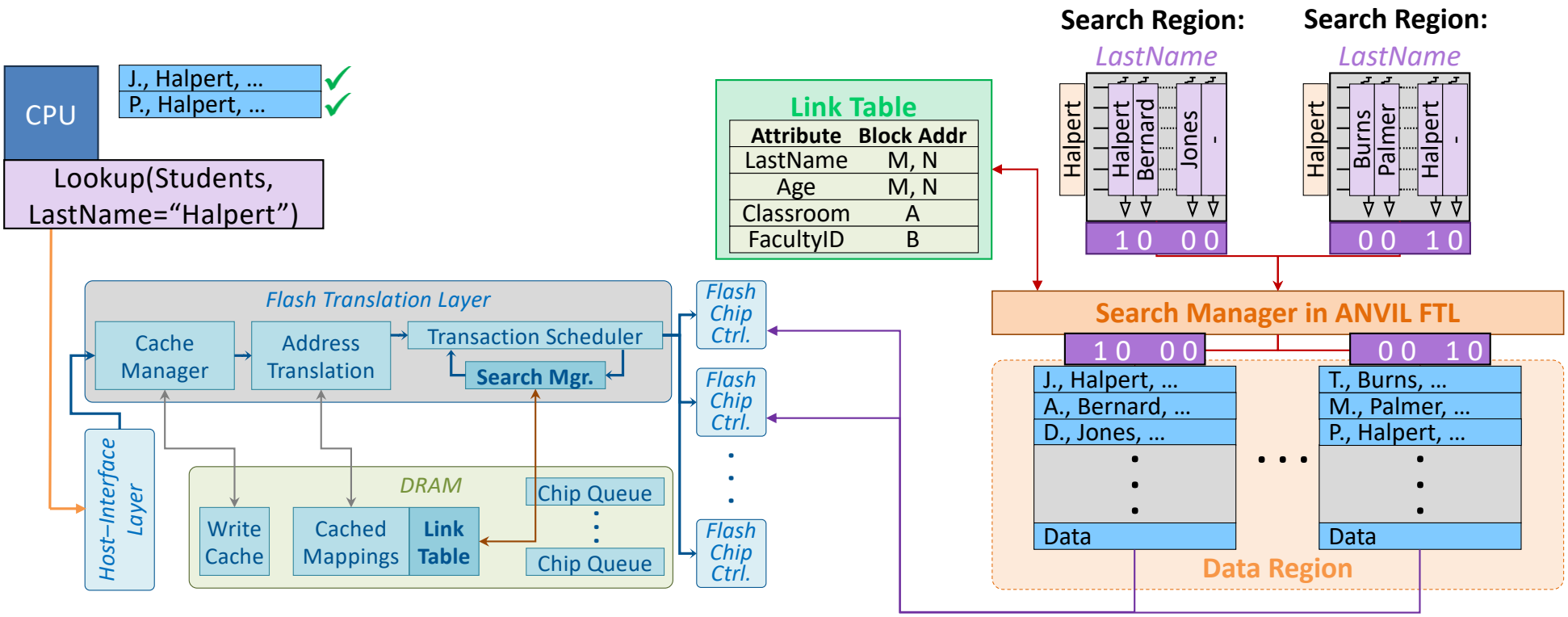
Conventional SSD Query

- Query: `SELECT {*} FROM {Students} WHERE LastName='Halpert';`
- Requires moving *all* six data elements even though *only* two are needed



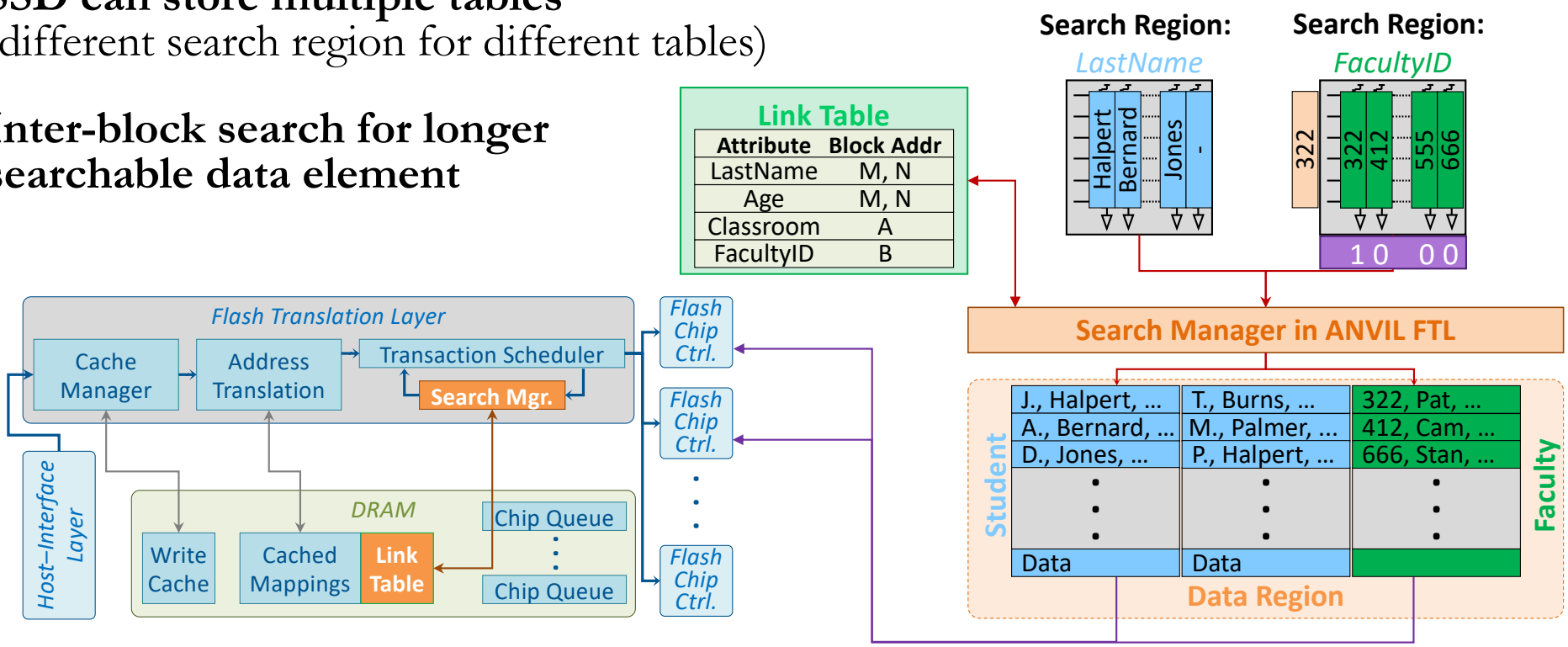
Lookup Operations With ANVIL

- Query: `SELECT {*} FROM {Students} WHERE LastName='Halpert';`



Supporting Multiple Tables

- **Link table can support multiple search regions mapped to a single data region**
- **SSD can store multiple tables**
  (different search region for different tables)
- **Inter-block search for longer searchable data element**



NVMe Interface

- **NVMe 2.0 I/O command set**
  - Preserves support for basic block I/O commands (e.g., RD/WR)
  - Allows vendors to selectively support ANVIL functionality

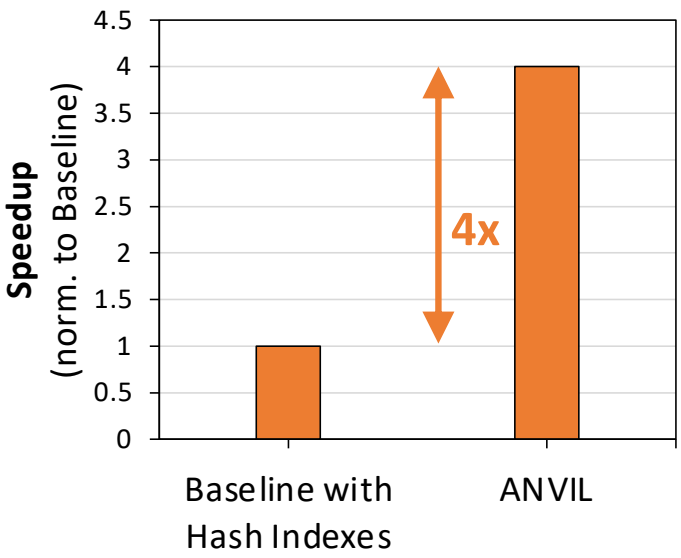
Command	Function
Allocate	Allocate search region
Append	Stores data to previously allocated region
Deallocate	Deallocate search region, mark for block erase
Simple Lookup	Look up simple name
Lookup	Look up using complex search name stored in DRAM
Lookup Continue	Returns data overflow from ANVIL Lookup
Delete	Search for name, delete all matches

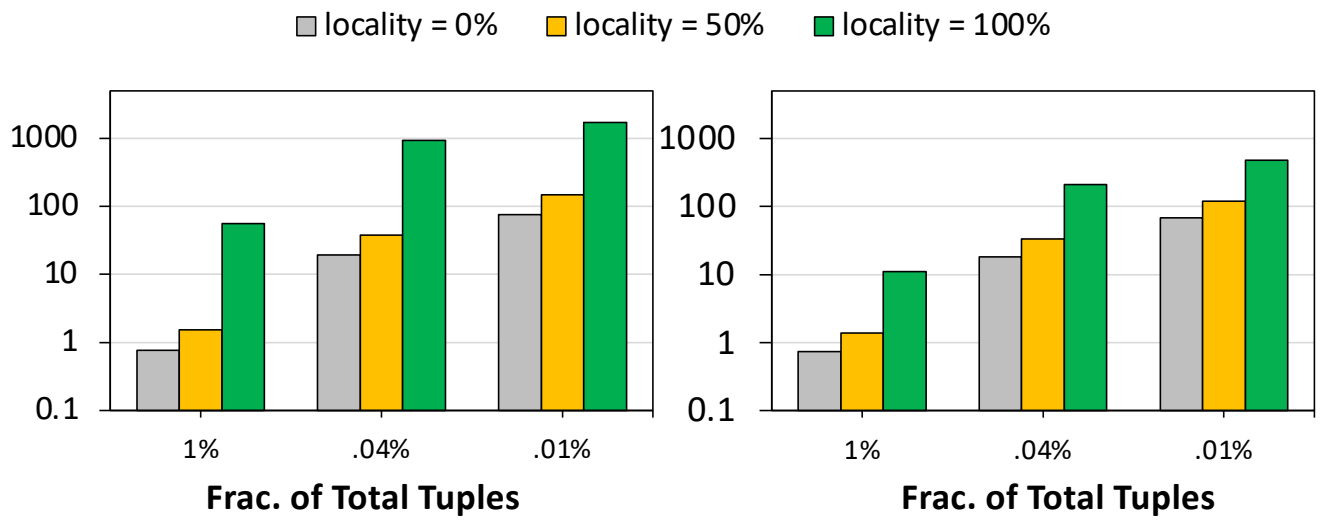
Evaluation

- **Evaluated using SimpleSSD and in-house analytical models**
- **Two 2TB SSDs with 3D NAND flash**
  - SSD-A: 8 channel, 8 dies/channel, 2 planes/die, 2048 blocks/plane, 196 pages/block, 16 kB pages
  - SSD-B: 4 channel, 4 dies/channel, 2 planes/die, 2048 blocks/plane, 96 pages/block, 16 kB pages
  - Dual-core in-order 4-stage CPU @ 800MHz (gem5)
- **Baseline assumes a CPU that sends Rd/Wr requests**
- **Three example NVP workloads**
  - **Online transaction processing (OLTP):** TPC-C database with hash indexes
  - **Online analytics processing (OLAP):** TPC-H generated using dbgen
  - **Graph analytics:** single-source shortest path (SSSP) with adjacency list
    - » Collection of real-world and synthetic graphs
    - » 1.1M-121.7M nodes; 1.5M-1.5B edges

OLTP/OLAP

- ANVIL improves performance by 4x for OLTP
- ANVIL improves performance by 480x-1568x for OLAP

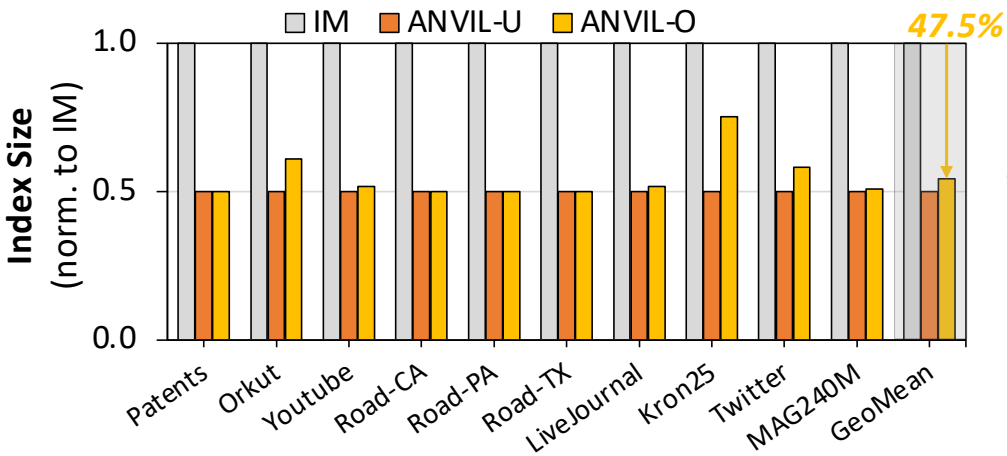


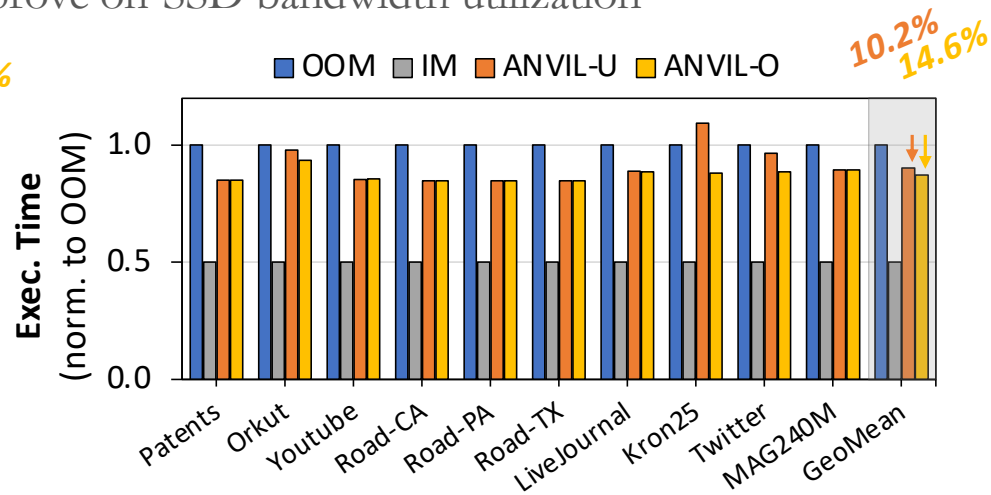


Speedups for both OLTP and OLAP-style queries

Graphs Analytics

- **ANVIL reduces main memory overheads**
  - IM holds the graph index completely in memory
  - OOM stores both the graph and index metadata in storage
  - ANVIL-U uses only *Lookup* and search regions
  - ANVIL-O combines *Lookup* with pointers to improve on-SSD bandwidth utilization

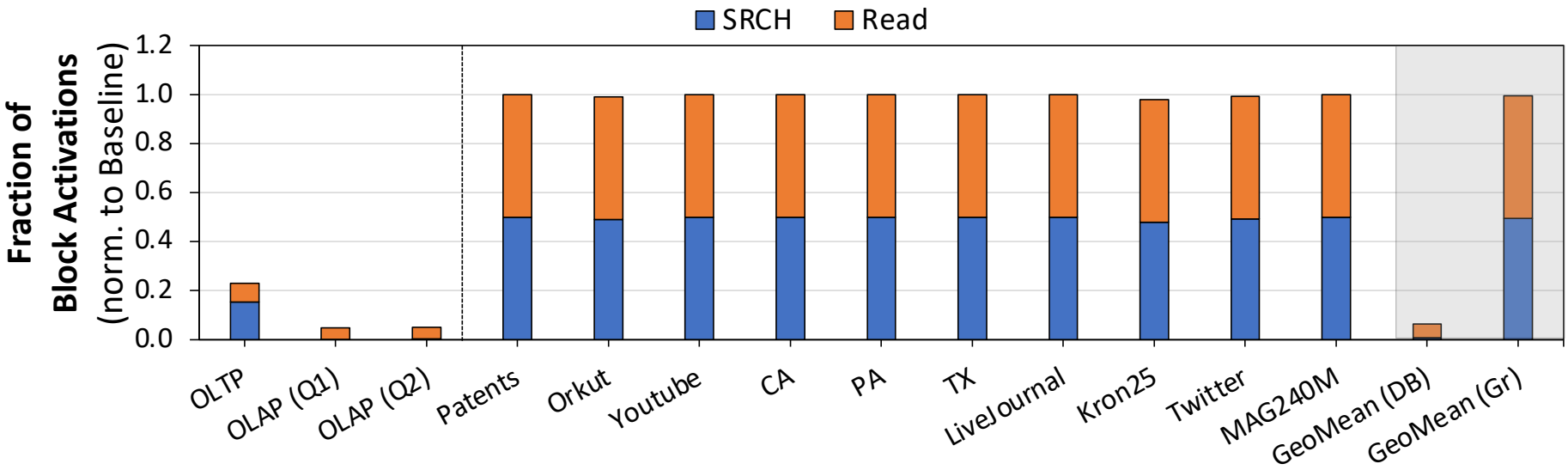




*Speedup of **14.6%** while reducing in-main-memory overheads by **47.5%***

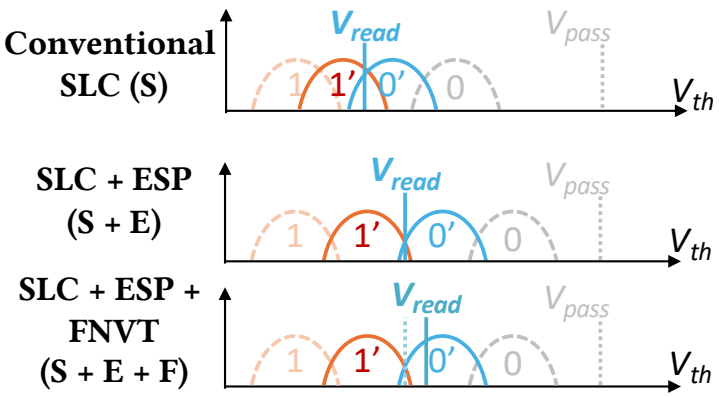
Energy

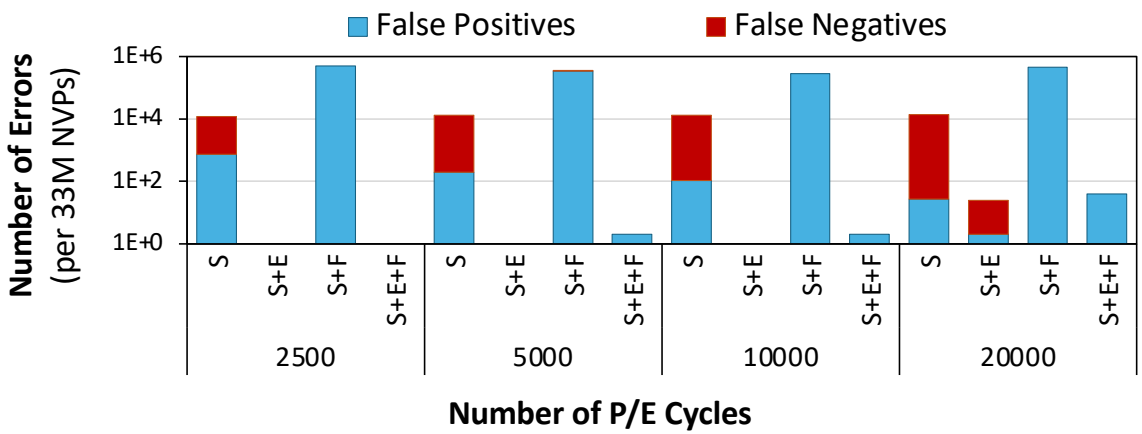
- **We use number of NAND flash blocks as rough approximation for energy**
  - SRCH applies half V_{read} and half V_{pass} per block
  - Read applies V_{read} and $(N-1)$ V_{pass} per block



Reliability & Errors

- **Leverage semantic meaning of *what* it means when an error occurs**
  - **False positive (FP):** NVP is *read* from data region, *can* be filtered by FTL or host CPU
  - **False negative (FN):** NVP is *not* retrieved, ***cannot*** be corrected
- **False-Negative Voltage Tuning (FNVT): dynamically increase V_{read}**
  - Compatible with Enhanced SLC Programming (E) [Park et al., MICRO '22]
  - Can eliminate false negatives after thousands of P/E cycles





S+E+F eliminates false negatives with only 0.2% firmware overhead

Optimizations & Further Analysis

- **Optimizations**
  - Early (Conditional) Termination
  - Write Inversion
  - Data Result Compaction
  - Duplicate Page Reads
- **Analysis of SSD internal bandwidth**
- **Workload characteristics**
  - Fused-name optimizations
  - Selectivity and locality
- **Update mechanisms to reduce flash writes**
- **Interactions with main memory**
- **Multi-SSD systems**

See paper for more information!

ANVIL Summary

- **Name–value pairs provide a flexible and convenient way to organize and capture data in many applications**
- **ANVIL accelerates name–value pair workloads *while still supporting general-SSD functionality***
  - Generic support for NVP enabled broad applicability across software and hardware structures
  - Provides acceleration and optimizations for Lookup functionality used in NVP workloads
  - Develops end-to-end support for applications via NVMe-compatible interface
- **Speedup: 4x for database transactions, 25x for database analytics, 14.6% speedup for graph workloads**

Introduction

ANVIL

(ISCA 2025)

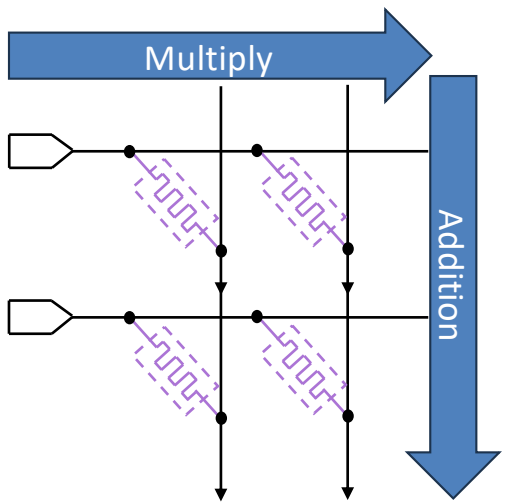
DARTH-PUM: A Preview

(ASPLOS 2026)

Digital PUM Can Do More Than MVM

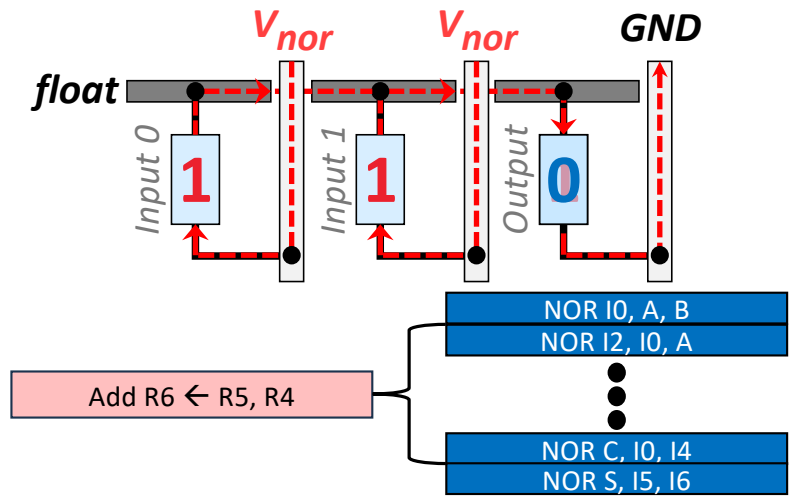
Analog PUM

High-throughput approximate matrix–vector multiplication



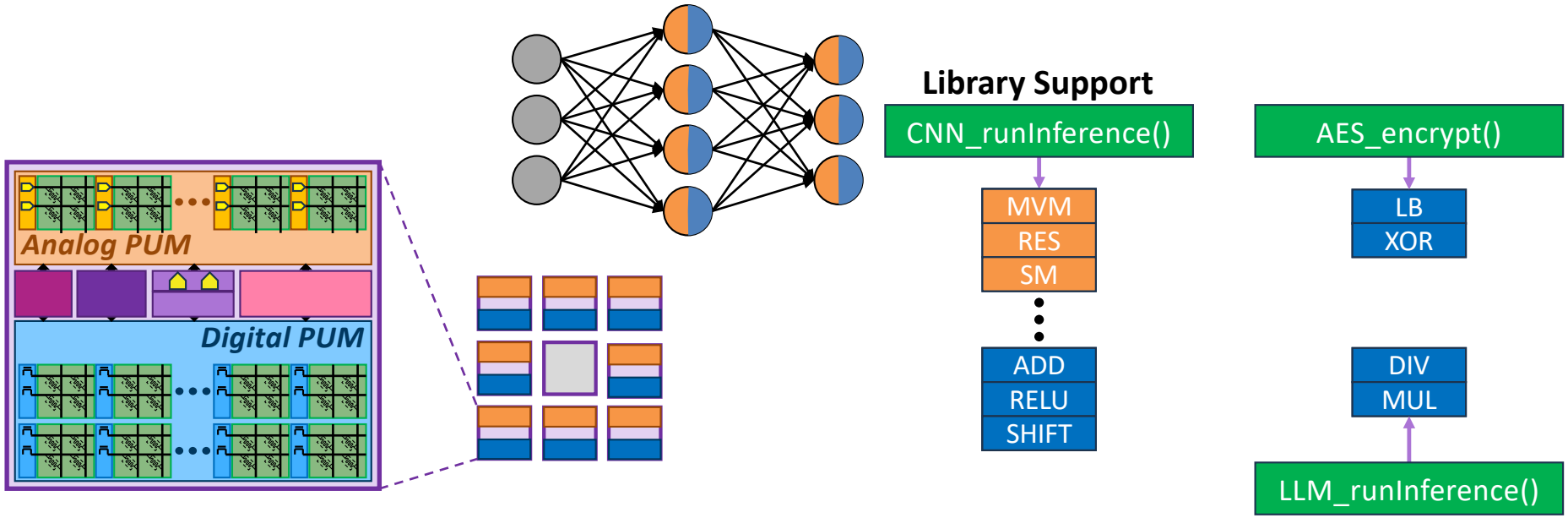
Digital PUM

Logically complete primitives for general-purpose compute



Many memories can support *both* analog *and* digital PUM
(e.g., ReRAM, MRAM, PCM, NAND/NOR flash)

DARTH-PUM: Digital–Analog Resistive Tiles for Hybrid PUM

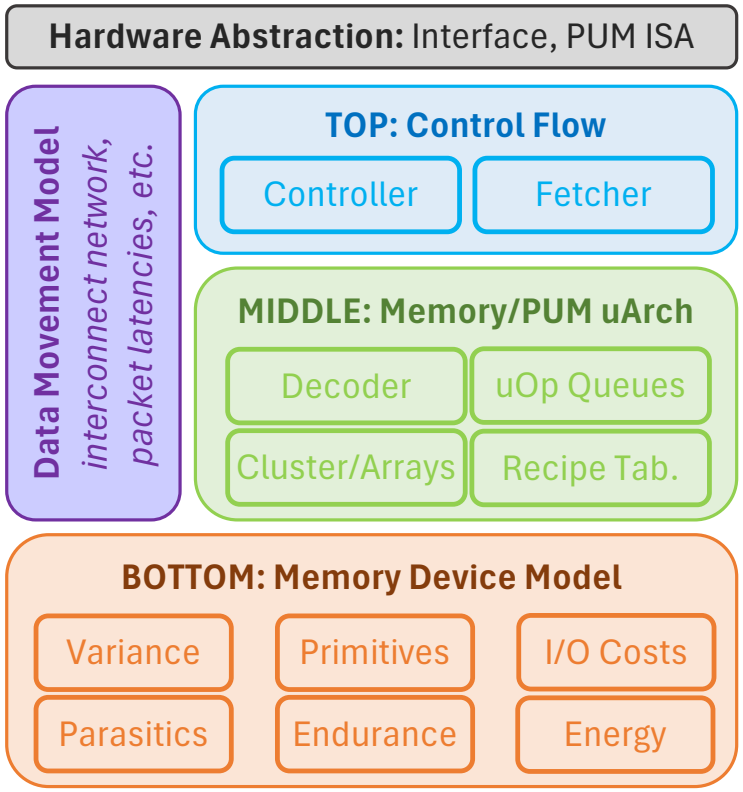


Analog+Digital PUM | **Eliminate Data Movement** | **ISA Support** | **Application-Specific Optimizations**

*DARTH-PUM improves throughput / energy for AES, ResNet-20, and LLM encoders by **39.6x / 66.8x** vs. CPU + Analog PUM*

MASTODON: An Exploratory Cross-Stack Simulator

- **Detailed estimation of**
  - **Performance:** latency and throughput
  - **Energy** and thermal dissipation
  - **Area** of architecture, including
    - » Arrays
    - » Peripherals
    - » Controllers
- **Unlike existing tools, we aim to**
  - Model memories with ideally ***any* device**
  - Directly take **device and circuit models** as an input
  - Explore **many different architectural configurations**
  - Show potential for
    - » Conventional memory
    - » Processing-using-memory (PUM)
- **Open Sourced:**
  https://github.com/ARCANA-Research/MASTODON



Conclusion

- Application space is wide with numerous different types of computations
- Generic abstractions and common kernels can be optimized
- Cross-stack evaluation necessary for realistic HW/SW co-design

ANVIL
(ISCA 2025)

DARTH-PUM
(ASPLOS 2026)

MASTODON
(GitHub)

DARTH-PUM is an exciting opportunity to integrate acceleration *and* general-purpose PUM!
(Come find out more at our talk on Wednesday (03/25) at 8:30a in Session 4D: PIM!!!)

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

Enabling Near-Accelerator Performance With Generic Processing-Using-Memory Architectures

ANVIL
(ISCA 2025)

DARTH-PUM
(ASPLOS 2026)

MASTODON
(GitHub)

Ryan Wong